Object-Oriented Programming

Naeemullah Khan

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

LMH
Lady Margaret Hall
University of Oxford

KAUST Academy
King Abdullah University of Science and Technology
Stage 1 · Course 3

June 13, 2026

Module 1
CLASSES & OBJECTS

The class keyword, constructors, variables, and identity

Module Content

This module covers the OOP paradigm and building Python classes from scratch — including constructors, variables, identity, and object representation.

- **Why OOP?**: bundling data and behaviour together.
- **Classes and objects**: the `class` keyword and instantiation.
- **Constructors**: `__init__` and `self`.
- **Variables**: instance variables vs. class variables.
- **Namespaces**: class vs. instance lookup order.
- **Identity**: `id()`, `is`, and `==`.
- **Representation**: `__str__` vs. `__repr__`.

Why Object-Oriented Programming?

Object-oriented programming is a way of organizing programs around **objects** that combine data with the behavior that operates on that data.

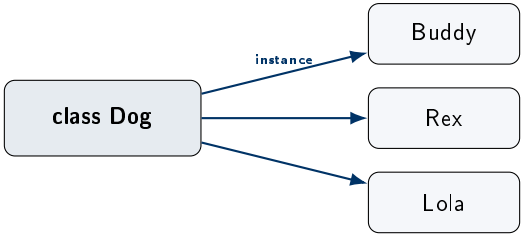
- Bundle related data and operations together.
- Reuse code through inheritance and composition.
- Hide complex details behind a simple interface.
- Model real things: students, accounts, neural-network layers.

The four pillars

- **Encapsulation**
- **Inheritance**
- **Polymorphism**
- **Abstraction**

Classes and Objects

A **class** is a blueprint. An **object** (or **instance**) is something built from that blueprint.



Key idea

The **class** describes structure and behavior once. Each **object** carries its own data but shares the same behavior defined by its class.

The class Keyword

We define a class with `class`. By convention, class names use **CapWords**.

```python
class Dog:
    """A simple Dog class."""
    species = "Canis familiaris"      # class variable

    def __init__(self, name, age):    # constructor
        self.name = name              # instance variable
        self.age  = age

    def bark(self):                   # instance method
        return f"{self.name} says woof!"

rex = Dog("Rex", 5)          # create an instance
print(rex.bark())            # Rex says woof!
print(rex.species)           # Canis familiaris
```

Vocabulary

- **Class**: `Dog`, the blueprint.
- **Instance**: `rex`, a specific dog created from the blueprint.
- **Attribute**: a piece of data on the class or instance (e.g. `name`).
- **Method**: a function defined inside a class.

The __init__ Constructor and self

The `__init__` method runs **automatically** every time a new object is created. Its job is to set up the instance's starting state.

```
class Point:
    def __init__(self, x, y):
        self.x = x      # bind data to THIS instance
        self.y = y

p1 = Point(1, 2)        # Python calls Point.__init__(p1, 1, 2)
p2 = Point(5, 7)
print(p1.x, p2.x)       # 1 5  -> independent state
```

What is self?

`self` is the **instance being worked on**. Python passes it as the first argument behind the scenes. It is not a keyword, only a strong convention.

Common mistake

Forgetting `self` as the first parameter is the number-one beginner error in Python OOP.

Instance Variables vs. Class Variables

Class variable

- Defined directly inside the class.
- **Shared** by every instance.
- Stored in the class itself.

Instance variable

- Defined inside `__init__` (or other methods).
- **Unique** to each instance.
- Stored in the instance itself.

```
class Counter:
    total = 0                       # CLASS variable (shared)

    def __init__(self, name):
        self.name = name            # INSTANCE variable
        Counter.total += 1

a = Counter("A")
b = Counter("B")
print(Counter.total)                # 2  -> shared across all
```

Rule of thumb

Use class variables for values that belong to the type as a whole. Use instance variables for values that belong to a specific object.

Class vs. Instance Namespaces

Every class and every instance has its own namespace (a dictionary of names). Python looks up an attribute first on the instance, then on the class.

```
class Dog:
    legs = 4                    # class namespace

rex = Dog()
print(rex.legs)                 # 4  -> looked up on the class
rex.legs = 3                    # creates an INSTANCE attribute
print(rex.legs)                 # 3  -> from instance namespace
print(Dog.legs)                 # 4  -> the class is unchanged

print(rex.__dict__)             # {'legs': 3}
print(Dog.__dict__.keys())      # dict_keys([..., 'legs', ...])
```

Mental model

Setting `rex.legs = 3` does **not** mutate the class. It adds a new key to the instance's own `__dict__`, which then shadows the class attribute for this instance only.

Object Identity: id(), is, and ==

Python distinguishes between two objects being the **same object** and two objects having **equal values**.

Operator / function	Meaning
`id(x)`	A unique integer for the object (its identity).
`a is b`	True when `a` and `b` are the **same** object.
`a == b`	True when `a` and `b` have **equal** values. Calls `__eq__`.

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

a == b      # True   -> same value
a is b      # False  -> different objects
a is c      # True   -> same object (alias)
```

Practical rule

Use `is` only when comparing with `None`, `True`, or `False`. For everything else, use `==`.

Representing Objects: `__str__` vs. `__repr__`

Python uses two methods to convert an object to text. They have different audiences.

`__str__`

- Used by `print()` and `str()`.
- For **end users**: friendly and short.

`__repr__`

- Used in the REPL and by `repr()`.
- For **developers**: precise and unambiguous.

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __repr__(self):
        return f"Point(x={self.x}, y={self.y})"
    def __str__(self):
        return f"({self.x}, {self.y})"

p = Point(3, 4)
print(p)          # (3, 4)              -> __str__
print(repr(p))    # Point(x=3, y=4)     -> __repr__
[p]               # [Point(x=3, y=4)]   -> repr inside containers
```

Module 2

METHODS, PROPERTIES & ENCAPSULATION

Methods, properties, privacy, and memory

Module Content

This module covers how objects expose and protect their data through methods, properties, and privacy conventions.

- **Three kinds of methods**: instance, class, and static.
- **Properties**: using `@property` for computed and validated attributes.
- **Read-only attributes**: preventing external modification.
- **Privacy conventions**: single _ vs. double __ name mangling.
- **__slots__**: restricting attributes for memory optimization.

Three Kinds of Methods

A class can hold three kinds of methods. They differ in what is automatically passed as the first argument.

```
class Pizza:
    radius_cm = 30                      # class attribute

    def __init__(self, toppings):
        self.toppings = toppings

    def describe(self):                 # INSTANCE method
        return f"Pizza with {self.toppings}"

    @classmethod
    def margherita(cls):                # CLASS method (factory)
        return cls(["mozzarella", "tomato", "basil"])

    @staticmethod
    def is_round():                     # STATIC method
        return True
```

- **Instance method**: receives `self`; works on a single object.
- **Class method**: receives `cls`; often used as an alternative constructor.
- **Static method**: receives nothing extra; a plain function placed inside the class for grouping.

Properties: @property

A **property** lets you compute or validate an attribute, while keeping the simple `object.attribute` syntax.

```python
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self):                    # getter
        return self._radius

    @radius.setter
    def radius(self, value):             # setter with validation
        if value < 0:
            raise ValueError("radius must be >= 0")
        self._radius = value

    @property
    def area(self):                      # computed, read-only
        return 3.14159 * self._radius ** 2

c = Circle(5)
print(c.area)            # 78.5  (no parentheses!)
c.radius = 10            # validated through the setter
```

Read-only Attributes

If you omit the setter, the property becomes **read-only** from outside the class.

```
class User:
    def __init__(self, username):
        self._username = username

    @property
    def username(self):
        return self._username        # no setter -> read-only

u = User("alice")
print(u.username)                    # alice
u.username = "bob"                   # AttributeError
```

Pythonic principle

Start with a plain attribute. Switch to a property **only when you need** validation, computation, or read-only behavior. The caller's syntax does not change, so this refactor is invisible from outside.

Privacy Conventions: _ vs. __

Python has **no truly private** attributes. It uses naming conventions instead.

```
class Account:
    def __init__(self, balance):
        self.owner    = "public"     # public attribute
        self._balance = balance      # "internal use only"
        self.__pin    = 1234         # name-mangled by Python

a = Account(500)
a._balance        # works, but you are saying "I know I shouldn't"
a.__pin           # AttributeError
a._Account__pin   # 1234   -> the actual mangled name
```

Convention	Meaning
`name`	Public attribute. Free to use.
`_name`	"Internal, hands-off." Linters warn but Python does not enforce.
`__name`	**Name-mangled** to `_ClassName__name`. Used to avoid clashes in subclasses, not for security.

__slots__: Memory Optimization

By default, every instance carries a `__dict__`. `__slots__` replaces this dictionary with a fixed array of attribute slots.

```python
class PointSlow:
    def __init__(self, x, y):
        self.x = x
        self.y = y

class PointFast:
    __slots__ = ("x", "y")
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = PointFast(1, 2)
p.z = 3   # AttributeError: only x, y allowed
```

Pros

- Smaller memory (40–50% less per instance).
- Faster attribute access.

Cons

- No new attributes after definition.
- Multiple inheritance becomes trickier.

When to use

Reach for `__slots__` when you create **millions** of small objects. For typical app code it is rarely needed.

Module 3
INHERITANCE & POLYMORPHISM

Class hierarchies, duck typing, and mixins

Module Content

This module covers building class hierarchies and writing flexible code that works across related types.

- **Single inheritance**: extending a class with `super()`.
- **MRO**: Python's method resolution order (C3 linearisation).
- **Multiple inheritance**: the diamond problem and how Python resolves it.
- **Abstract base classes**: enforcing interface contracts.
- **Polymorphism**: writing code that works for any compatible type.
- **Duck typing**: if it walks like a duck...
- **Mixins**: adding orthogonal behaviour through inheritance.
- **Type checking**: `isinstance()` and `issubclass()`.

Single Inheritance and super()

A **child class** reuses behavior from a **parent class** and may add or override methods.

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "Some sound"

class Dog(Animal):                          # Dog INHERITS from Animal
    def __init__(self, name, breed):
        super().__init__(name)              # call parent's __init__
        self.breed = breed
    def speak(self):                        # OVERRIDE
        return f"{self.name} barks!"

d = Dog("Rex", "Labrador")
print(d.speak())                            # Rex barks!
print(isinstance(d, Animal))                # True
```

Why super()?

`super()` delegates to the next class in the lookup order. It is safer than naming the parent directly, especially when classes are later reorganized.

Method Resolution Order (MRO)

When a method is called, Python searches classes in a specific order called the **Method Resolution Order**. It uses the **C3 linearization** algorithm.

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass

print(D.__mro__)
# (D, B, C, A, object)
```

Two C3 guarantees

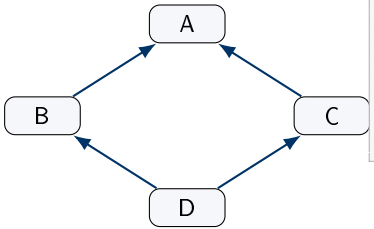
- A subclass appears **before** its parents.
- The relative order of parents is preserved.

Inspecting the MRO

Use `ClassName.__mro__` or `ClassName.mro()` to see the exact lookup order. Every `super()` call follows this list.

Multiple Inheritance and the Diamond Problem

A class can inherit from more than one parent. When two parents share a common ancestor, we get a **diamond** shape.




```
class A:
    def hi(self): print("A")
class B(A):
    def hi(self): print("B"); super().hi()
class C(A):
    def hi(self): print("C"); super().hi()
class D(B, C):
    def hi(self): print("D"); super().hi()

D().hi()       # D B C A
```

What C3 prevents

Without a careful algorithm, calling `D().hi()` could call `A` twice or skip a class entirely. C3 produces a single, predictable order: `D, B, C, A`.

Abstract Base Classes

An **abstract base class** (ABC) declares methods that subclasses **must** implement.

```python
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        ...
    @abstractmethod
    def perimeter(self):
        ...

class Square(Shape):
    def __init__(self, side):
        self.side = side
    def area(self):       return self.side ** 2
    def perimeter(self):  return 4 * self.side

Shape()                 # TypeError: can't instantiate abstract class
print(Square(3).area())  # 9
```

Why use abstract classes?

They make the **contract** explicit. Anyone subclassing `Shape` is forced to provide `area` and `perimeter`, so the rest of the program can rely on them.

Polymorphism and Duck Typing

Polymorphism means the same call works on different types. Python takes the loose, flexible version called **duck typing**.

```
class Duck:
    def speak(self): return "Quack"
class Robot:
    def speak(self): return "Beep"
class Cat:
    def speak(self): return "Meow"

def chorus(creatures):
    for c in creatures:           # don't care about the type
        print(c.speak())          # only the interface matters

chorus([Duck(), Robot(), Cat()])
```

The Pythonic credo

"If it walks like a duck and quacks like a duck, it is a duck." Java and C++ require a shared base class. Python just calls `.speak()` and trusts you.

Mixins: Composition through Inheritance

A **mixin** is a small class meant to be combined with others to add a focused piece of behavior.

```
class JSONMixin:
    def to_json(self):
        import json
        return json.dumps(self.__dict__)

class TimestampMixin:
    def stamp(self):
        from datetime import datetime
        self.created_at = datetime.now()

class User(JSONMixin, TimestampMixin):
    def __init__(self, name):
        self.name = name
        self.stamp()

u = User("Ada")
print(u.to_json())
```

Where you have seen this

Mixins are used heavily in **Django** (e.g. `LoginRequiredMixin`) and in **Keras** training-loop classes. Each mixin contributes one orthogonal feature.

Type Checking: isinstance() and issubclass()

`isinstance(obj, T)` checks whether an object is of type `T` (or a subclass). `issubclass(C, P)` checks whether one class inherits from another.

```
isinstance(5, int)              # True
isinstance(5, (int, float))     # True   -> tuple of types
isinstance(True, int)           # True   -> bool IS an int!

issubclass(bool, int)           # True
issubclass(Dog, Animal)         # True
```

Use sparingly

Heavy use of `isinstance` often signals you should be using **polymorphism** instead. Reserve type checks for boundaries: input validation, dispatching, and library entry points.

Module 4
OOP DESIGN PRINCIPLES

Encapsulation, SOLID, composition, and dataclasses

Module Content

This module covers industry best practices for writing clean, extensible, and maintainable object-oriented code.

- **Encapsulation**: hiding the "how" behind a clean interface.
- **SOLID principles**: an overview of the five design guidelines.
- **Composition over inheritance**: favouring has-a over is-a.
- **Dataclasses**: eliminating boilerplate with `@dataclass`.
- **Named tuples vs. dataclasses**: choosing the right data container.

Encapsulation: Hiding the "How"

Encapsulation means exposing **what** an object does, while hiding **how** it does it.

- Keep internal state behind _ or properties.
- Offer a stable public interface.
- Validate, log, or cache transparently inside setters.
- Refactor internals freely without breaking callers.

Self-test

Could I rewrite the internals of this class tomorrow without changing any of its callers? If yes, the class is well-encapsulated.

Key idea

Encapsulation is the discipline behind every other principle in this section. It is what lets us change code with confidence.

SOLID Principles (Overview)

SOLID is a set of five design principles for writing maintainable object-oriented code.

Principle	In one sentence
S — Single Responsibility	A class should have **one** reason to change.
O — Open/Closed	Code should be open for **extension**, closed for **modification**.
L — Liskov Substitution	A subclass must be usable wherever its parent is expected.
I — Interface Segregation	Many small interfaces beat one fat interface.
D — Dependency Inversion	Depend on abstractions, not on concrete implementations.

Today's focus

We focus on the first two: **S** (each class has one job) and **O** (extend by subclassing or composing, do not edit working code). The other three are explored in larger software-engineering courses.

Composition over Inheritance

Inheritance models an **is-a** relationship. **Composition** models a **has-a** relationship.

```
# Inheritance can become rigid
class FlyingCar(Car, Plane):
    ...

# Composition is flexible: a Car HAS an engine and HAS wheels
class Engine:
    def start(self): ...
class Wheels:
    def roll(self):  ...

class Car:
    def __init__(self):
        self.engine = Engine()
        self.wheels = Wheels()
    def drive(self):
        self.engine.start()
        self.wheels.roll()
```

Why prefer composition

Easier to test, easier to swap parts, and easier to extend. Use inheritance only when there is a genuine **is-a** relationship.

Dataclasses: Boilerplate-free Classes

The `@dataclass` decorator generates `__init__`, `__repr__`, and `__eq__` for you, based on type-annotated fields.

```python
from dataclasses import dataclass, field

@dataclass
class Product:
    name: str
    price: float
    tags: list = field(default_factory=list)
    in_stock: bool = True

p = Product("Book", 19.99)
print(p)
# Product(name='Book', price=19.99, tags=[], in_stock=True)

p == Product("Book", 19.99)         # True   -> auto __eq__
```

Useful flags

`frozen=True` makes instances immutable. `order=True` adds comparison operators. `slots=True` (Python 3.10+) adds `__slots__` automatically.

Named Tuples vs. Dataclasses

Both build small data containers. They differ in mutability and flexibility.

NamedTuple

```python
from typing import NamedTuple

class Point(NamedTuple):
    x: float
    y: float

p = Point(1, 2)
p[0]      # tuple-like access
p.x       # attribute access
# IMMUTABLE, hashable
```

Dataclass

```python
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float

p = Point(1, 2)
p.x = 5     # mutable by default
# Add methods freely
```

Choosing the right tool

- Pick **NamedTuple** for small, immutable, tuple-like records.
- Pick **dataclass** when you need mutability, defaults, or extra methods.

Module 5

MAGIC METHODS & INTROSPECTION

Dunders, operator overloading, and runtime inspection

Module Content

This module unlocks Python's operator overloading system and covers inspecting objects at runtime.

- **Dunder methods**: how Python calls them automatically with built-in syntax.
- **Arithmetic dunders**: `__add__`, `__mul__`, and friends.
- **Comparison dunders**: `__eq__`, `__lt__`, and total ordering.
- **Container dunders**: `__len__`, `__getitem__`, and iteration.
- **Context manager dunders**: `__enter__` and `__exit__`.
- **Callable objects**: `__call__`.
- **Introspection**: `__dict__`, `__class__`, and `__slots__`.
- **Worked example**: building a Matrix class end-to-end.

Magic Methods (Dunders)

Dunder methods are methods with double-underscore names. Python calls them automatically when you use built-in syntax. Implementing them makes your class **feel like a built-in type**.

Syntax you write	Method Python calls
`a + b`	`a.__add__(b)`
`a == b`	`a.__eq__(b)`
`len(a)`	`a.__len__()`
`a[i]`	`a.__getitem__(i)`
`for x in a`	`a.__iter__()`
`a()`	`a.__call__()`
`with a:`	`a.__enter__` / `__exit__`

Arithmetic Dunders

Define arithmetic dunders to use +, -, *, /, and ** on your objects.

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, o):     return Vector(self.x + o.x, self.y + o.y)
    def __sub__(self, o):     return Vector(self.x - o.x, self.y - o.y)
    def __mul__(self, k):     return Vector(self.x * k,   self.y * k)
    def __truediv__(self, k): return Vector(self.x / k,   self.y / k)
    def __pow__(self, n):     return Vector(self.x ** n,  self.y ** n)
    def __repr__(self):       return f"Vector({self.x}, {self.y})"

v = Vector(1, 2) + Vector(3, 4)      # Vector(4, 6)
w = Vector(2, 3) * 5                 # Vector(10, 15)
```

Reflected operations

For `5 * v` (where `v` is a Vector), Python tries `int.__mul__` first, then falls back to `Vector.__rmul__`. Define the reflected versions when needed.

Comparison Dunders

Comparison dunders make your objects work with <, <=, ==, >=, >, and with `sorted()`.

```
from functools import total_ordering

@total_ordering
class Money:
    def __init__(self, amount):
        self.amount = amount
    def __eq__(self, o):
        return self.amount == o.amount
    def __lt__(self, o):
        return self.amount < o.amount
    # __le__, __gt__, __ge__ filled in by @total_ordering

a, b = Money(10), Money(20)
print(a < b)      # True
print(a >= b)     # False
print(sorted([Money(30), Money(10), Money(20)]))
```

Tip

Implement `__eq__` and `__lt__`, then let `@total_ordering` fill in the rest. You write less code and get consistent behavior.

Container Dunders

Implementing container dunders makes your class behave like a list, dict, or set.

```
class Playlist:
    def __init__(self, songs):      self.songs = list(songs)
    def __len__(self):              return len(self.songs)
    def __getitem__(self, i):       return self.songs[i]
    def __setitem__(self, i, v):    self.songs[i] = v
    def __delitem__(self, i):       del self.songs[i]
    def __contains__(self, s):      return s in self.songs
    def __iter__(self):             return iter(self.songs)

pl = Playlist(["A", "B", "C"])
len(pl)                 # 3
pl[0]                   # 'A'
"B" in pl               # True
for song in pl:         # iterates
    print(song)
```

The pattern

Decide which built-in operations make sense for your class, then implement the matching dunders. Users get a familiar, Pythonic interface for free.

Context Manager Dunders

`__enter__` and `__exit__` let your class be used in a `with` block. They are perfect for resources that need setup and cleanup.

```
class Timer:
    def __enter__(self):
        from time import perf_counter
        self.t = perf_counter()
        return self                          # bound to ‘as‘ variable
    def __exit__(self, exc_type, exc, tb):
        from time import perf_counter
        self.elapsed = perf_counter() - self.t
        print(f"Took {self.elapsed:.3f}s")
        return False                         # don’t suppress exceptions

with Timer() as t:
    sum(i * i for i in range(10**6))
# Took 0.058s
```

Why this matters

`__exit__` runs **even if** the block raises an exception, so cleanup always happens. This is the same mechanism used by `open()` and database connections.

Callable Objects: `__call__`

Define `__call__` to make instances behave like functions. This is useful for "functions with state."

```python
class Multiplier:
    def __init__(self, factor):
        self.factor = factor
    def __call__(self, x):                # makes instances callable
        return x * self.factor

double = Multiplier(2)
triple = Multiplier(3)

print(double(5))          # 10
print(triple(5))          # 15
print(callable(double))   # True
```

Where you have seen this

PyTorch's `nn.Module` layers are callable. So are scikit-learn transformers. Decorators that take arguments often use this pattern as well.

Introspection: __dict__, __class__, __slots__

Python exposes several attributes that let you inspect any object at runtime.

```
class Car:
    wheels = 4
    def __init__(self, model):
        self.model = model

c = Car("Tesla")

c.__dict__                # {'model': 'Tesla'}     instance attributes
Car.__dict__              # mappingproxy with 'wheels', '__init__', ...
c.__class__               # <class '__main__.Car'>  the class of c
type(c) is c.__class__    # True

# With __slots__:
class Pt:
    __slots__ = ("x", "y")
Pt().__dict__             # AttributeError -> no per-instance dict
```

Why introspection helps

Debugging, serialization (e.g. converting to JSON), and frameworks like Django use these attributes to discover your class structure automatically.

Thank you!

Questions and discussion

Course takeaway

The goal of OOP is not to use every feature, but to write classes that are clear, reusable, and feel natural to other Python programmers.